

TITLE OF THE THESIS PAPER

Candidate: Given Name FAMILY NAME

Scientific coordinator: Professor/Associate professor/Assistant professor/Lecturer/Teaching assistant Given Name FAMILY NAME

Session: June 2023

ABSTRACT

The abstract is intended to inform about the content of the paper through a brief description of the research of up to one page, the procedures/methods, as well as its results or conclusions. The abstract in Romanian becomes mandatory for works edited in languages other than Romanian and will be written in 12 pt Nimbus Sans fonts. It will start two blank lines after the heading "ABSTRACT". Before the title, there will be three blank 12 pt. lines.

ABSTRACT

The English abstract will be on the third page of the manuscript and will present synthetically the paper work. The maximum length of the abstract is one page written with Nimbus Sans characters, size 12 pt. The abstract text will begin after two blank lines (size 12pt.) from the “ABSTRACT” title. Before the title there will be left three blank lines of 12pt size.

CONTENTS

1	Introduction	6
1.1	Approached Domains	6
1.2	Context and Motivation	6
1.3	General Specifications	7
1.4	Main Results	7
1.5	Structure of the Project	7
2	State of the Art	9
2.1	PID Control	9
2.2	Fuzzy Logic and Fuzzy Control	9
2.3	Depth-First Search for Maze Solving	10
3	General Description	11
3.1	System Overview	11
3.2	Hardware Configuration	11
3.3	Software Architecture	12
4	Implementation	13
4.1	The Lane Keeping Module	13
4.1.1	Task Description	13
4.1.2	The Lane Keeping Controller	14
4.1.3	PID Algorithm	18
4.1.4	Fuzzy PI Algorithm	20
4.1.5	Fuzzy Rule-Based Algorithm	25
4.2	The Maze Solver Module	35
4.2.1	Task Description	35
4.2.2	The MazeMap Class	35
4.2.3	The MazeSolverController	36
4.2.4	Main Execution Loop	36
4.3	The Telemetry Interface	37
4.3.1	WifiStreamer	37
4.3.2	PC Receiver and Visualization	37
4.4	Object-Oriented Architecture	37
4.4.1	Class Hierarchy	37
4.4.2	SOLID Principles	38
4.4.3	Factory Pattern	38

5	Experimental Results	39
5.1	Experimental Setup	39
5.2	Lane Keeping Results	39
5.2.1	Disturbance Over Time	39
5.2.2	Performance Comparison	40
5.2.3	Turn Rate Output	40
5.2.4	Rule Activation Analysis	40
5.3	Maze Solver Results	40
5.3.1	Exploration Map	40
5.3.2	Speed Regulation	42
5.3.3	Maze Solver Performance	42
6	Conclusions and Perspectives	43
6.1	Conclusions	43
6.2	Synthesis of Contributions	43
6.3	Perspectives for Development	44

LIST OF FIGURES

3.1	High-level system architecture showing the data flow between the hardware layer, the control layer (with interchangeable algorithms), and the telemetry layer.	11
4.1	Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.	22
4.2	Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.	22
4.3	Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.	28
4.4	Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.	29
4.5	Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.	30
4.6	UML class diagram showing the Algorithm and Controller hierarchies.	37
5.1	Disturbance over time for the three algorithms in the lane keeping task.	39
5.2	Turn rate output over time for the three algorithms.	40
5.3	Rule activation frequency for the Fuzzy Rule-Based controller.	41
5.4	Maze exploration path showing the DFS traversal order.	41
5.5	Forward speed regulation during cell-to-cell driving.	42

LIST OF TABLES

4.1	Fuzzy Rule-Based: output labels and their crisp turn rate values.	31
4.2	Fuzzy Rule-Based: rule matrix for lane keeping.	31
4.3	Fuzzy Rule-Based: full IF-THEN rule set with physical meaning.	32
4.4	Maze solver configuration parameters.	35
4.5	Direction encoding and coordinate changes.	36
5.1	Lane keeping performance comparison (average of 5 runs).	40
5.2	Maze solver performance comparison.	42

LIST OF SNIPPETS

1. INTRODUCTION

1.1 APPROACHED DOMAINS

This project lies at the intersection of three engineering domains: *embedded control systems*, *fuzzy logic* and *autonomous mobile robotics*. The physical platform is the LEGO Mindstorms EV3, a widely used educational robot that provides servomotors, colour sensors, ultrasonic distance sensors and a gyroscope — all programmable through the Pybricks MicroPython environment.

The control problem is formulated as the reduction of a measured *disturbance* (the deviation from an ideal state) by computing a corrective output at each control cycle. Three families of controllers are explored: a classical PID controller, a Fuzzy PI controller that adapts its gains in real time, and a pure Fuzzy Rule-Based controller that maps the disturbance and its trend directly to the motor command through a Mamdani-type inference system. Each family is applied to two distinct tasks — lane keeping and maze solving — demonstrating that the same algorithmic structure can regulate fundamentally different physical quantities (turn rate and forward speed) when configured with appropriate parameters.

1.2 CONTEXT AND MOTIVATION

Previous diploma projects at UPT involving the LEGO Mindstorms EV3 focused on single-line following, where a single colour sensor tracked a black line on a white surface and the controller corrected the steering whenever the sensor reading deviated from the expected value. While effective as an introductory exercise, this approach has two limitations: the robot cannot navigate a bounded lane (two-sided track), and the controller gains are fixed regardless of how far the robot has drifted.

This project extends the problem in two directions. First, the lane keeping task replaces the single-line follower with a *dual-sensor, dual-boundary* configuration in which the robot drives between two parallel black lines on a white surface. The disturbance is defined as the difference between the left and right sensor readings, and the controller must keep this difference close to zero. Second, the maze solving task introduces a grid-based labyrinth that the robot must explore autonomously using a Depth-First Search algorithm, while the same control algorithms regulate the forward approach speed.

The motivation for introducing fuzzy logic is the observation that a fixed-gain PID controller applies the same proportional response regardless of whether the robot is 1 mm or 10 mm off-centre. A human driver, by contrast, would steer gently near the centre and aggressively near the edge. Fuzzy logic provides a formal framework to express this graduated behaviour, and comparing the three control strategies quantitatively is the central contribution of this work.

1.3 GENERAL SPECIFICATIONS

The project meets the following specifications:

- Two autonomous tasks: lane keeping between two black lines and maze solving in a grid-based labyrinth.
- Three interchangeable control algorithms: PID, Fuzzy PI and Fuzzy Rule-Based, selectable at startup through a single configuration variable.
- Real-time WiFi telemetry streaming of the control variables (disturbance, output, active rules) to a PC for live monitoring and post-run analysis.
- Object-oriented, SOLID-compliant software architecture in Pybricks MicroPython, allowing new algorithms or tasks to be added without modifying existing code.
- Quantitative evaluation through metrics: trajectory smoothness, crossing count, average speed, and rule activation statistics.

1.4 MAIN RESULTS

The experimental evaluation shows that the Fuzzy Rule-Based controller achieves the smoothest trajectory in the lane keeping task, with fewer oscillations than the PID controller and faster recovery from disturbances than the Fuzzy PI controller. The trend input allows the Fuzzy Rule-Based controller to moderate its corrections when the robot is already recovering, which eliminates the overshoot oscillations characteristic of the PID. In the maze solver, all three algorithms successfully regulate the forward speed, with the PID offering the most predictable deceleration curve and the Fuzzy Rule-Based providing the most adaptive response to variable corridor lengths.

The DFS-based maze solver successfully explores every reachable cell in the test maze and returns to the starting position, using gyroscope-guided turning and side-sensor corridor centering to maintain alignment within the grid.

1.5 STRUCTURE OF THE PROJECT

The remainder of this work is organized as follows:

- **Chapter 2** presents the theoretical background for PID control and fuzzy logic systems, with references to the foundational literature.
- **Chapter 3** provides a high-level system architecture diagram and describes the information flow between the robot, the control algorithms, and the telemetry interface.
- **Chapter 4** is the core of this work. It documents the implementation of each software module: the lane keeping controller (Section 4.1), the maze solver (Section 4.2), the telemetry interface (Section 4.3), and the object-oriented architecture (Section 4.4).

- **Chapter 5** presents the experimental results: comparative plots, performance tables and analysis of the three control strategies.
- **Chapter 6** summarizes the contributions, draws conclusions, and outlines directions for future development.

2. STATE OF THE ART

2.1 PID CONTROL

The Proportional-Integral-Derivative (PID) controller is the most widely deployed feedback controller in industrial and educational robotics **astrom2006**. Its output is a weighted sum of three terms: the proportional term reacts to the present error, the integral term eliminates steady-state offset by accumulating past errors, and the derivative term dampens oscillations by anticipating future error from the current rate of change **ogata2010**. The standard formulation is:

$$u(t) = K_P \cdot e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt} \quad (2.1)$$

where $e(t)$ is the error (setpoint minus measured value) and K_P , K_I , K_D are the tuning gains. The discrete-time implementation used in this project replaces the integral with a running sum and the derivative with a first-order backward difference, both gated by the time step Δt to maintain physical consistency.

A known limitation of PID controllers is that the gains are fixed: the same K_P applies whether the error is small or large. This produces a linear response characteristic that can cause oscillations near the setpoint (if gains are tuned for large errors) or sluggish correction far from the setpoint (if gains are tuned for stability).

2.2 FUZZY LOGIC AND FUZZY CONTROL

Fuzzy logic, introduced by Zadeh in 1965 **zadeh1965**, generalizes Boolean logic by allowing truth values between 0 and 1. A crisp input value is mapped to one or more fuzzy sets through *membership functions*, producing a degree of membership $\mu \in [0, 1]$ that quantifies how well the input matches each linguistic term (e.g. “Small”, “Medium”, “Large”).

A Mamdani-type fuzzy controller **mamdani1975** operates in three stages:

1. **Fuzzification:** the crisp input is converted into membership degrees using triangular or trapezoidal membership functions.
2. **Inference:** a set of IF-THEN rules maps the fuzzy input to a fuzzy output. The AND operator is implemented as the minimum of the input memberships, and multiple rules activating the same output label are aggregated using the MAX operator.
3. **Defuzzification:** the aggregated fuzzy output is converted back to a crisp value, typically using the centre-of-gravity (weighted average) method.

Fuzzy PI controllers, which use fuzzy logic to adapt the gains of a conventional PI controller, have been applied to mobile robot navigation by several authors **lee1990**, **passino1998**. The advantage over a fixed-gain PID is the ability to encode expert knowledge

(“if the error is small, use gentle corrections”) without requiring a precise mathematical plant model.

2.3 DEPTH-FIRST SEARCH FOR MAZE SOLVING

Depth-First Search (DFS) is a graph traversal algorithm that explores as far as possible along each branch before backtracking **cormen2009**. Applied to a grid-based maze, DFS treats each cell as a node and each open passage as an edge. The algorithm maintains a stack of visited cells: when no unvisited neighbour is available, it pops the stack to return to the most recent junction with unexplored branches. DFS guarantees complete exploration of any connected graph and requires only $O(V)$ memory, where V is the number of cells — well within the constraints of the EV3 MicroPython environment.

3. GENERAL DESCRIPTION

3.1 SYSTEM OVERVIEW

The system consists of three layers: the *hardware layer* (sensors, motors, EV3 brick), the *control layer* (algorithms and task controllers), and the *telemetry layer* (WiFi streaming to PC). Figure 3.1 presents the high-level architecture.

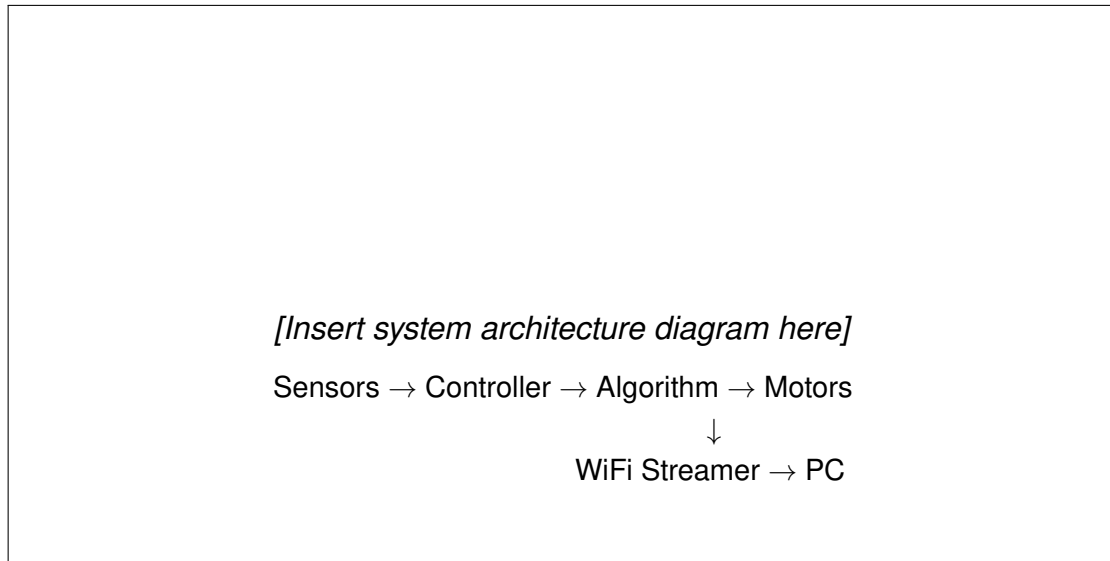


Figure 3.1: High-level system architecture showing the data flow between the hardware layer, the control layer (with interchangeable algorithms), and the telemetry layer.

3.2 HARDWARE CONFIGURATION

The robot is built from the LEGO Mindstorms EV3 Education Set. The drive train uses two large servomotors on ports A and C, connected through a differential gear to wheels with a diameter of 55.5 mm and an axle track of 104 mm. The Pybricks `DriveBase` class uses these dimensions to convert speed (mm/s) and turn rate (deg/s) commands into individual motor speeds.

For the lane keeping task, two colour sensors are mounted on ports S2 and S3, positioned symmetrically on the front of the robot to straddle the white lane between the two black boundary lines.

For the maze solving task, three ultrasonic sensors (forward on S4, left on S1, right on S2), a gyroscope (S3), and a sensor motor (B) that can rotate the forward sensor are used to perceive the maze and maintain accurate heading.

3.3 SOFTWARE ARCHITECTURE

The software is organized around the following abstractions:

- **Configuration:**
a single function `initializeConfiguration(task, algorithm)` generates a dictionary of all parameters for the selected task-algorithm pair.
- **Algorithm:** an abstract base class with a `compute(setpoint, measured, dt)` interface. Three concrete implementations (PID, Fuzzy PI, Fuzzy Rule-Based) are interchangeable at runtime.
- **Controller:** an abstract base class with a `run()` method. Two concrete controllers (LaneKeepingController, MazeSolverController) use the Algorithm interface to regulate the robot.
- **WifiStreamer:** a UDP socket wrapper that sends telemetry packets (step, disturbance, output) to a PC for real-time monitoring.

This architecture follows the SOLID principles: the Algorithm hierarchy satisfies the Open/Closed and Liskov Substitution principles (new algorithms can be added without modifying controllers), the single-responsibility principle separates sensing, computation and actuation, and the dependency inversion principle ensures that controllers depend on the abstract Algorithm interface rather than on concrete implementations.

4. IMPLEMENTATION

4.1 THE LANE KEEPING MODULE

4.1.1 TASK DESCRIPTION

This project implements a robust algorithm in which the LEGO Mindstorms EV3 follows a trajectory on a white surface that is bordered with the help of two black lines between which the robot is situated, thus keeping itself in a lane.

Previous diploma projects including Mindstorms EV3 included the task of following a single black line. The purpose of the robot was to make correction turns when the sensor attached to it began to read another value other than the one of the black line. With the help of proportional, integral and derivative gains, the turns are adjusted from gentle turns to harsh ones.

So, unlike traditional robots that follow a single line this project focuses on a robot that uses two color sensors to balance itself inside the white “road” that is contrasted with the two black edges. It utilizes a custom controller combined with an emergency safety net to achieve a reliable and smooth navigation.

The robot is evaluating continuously its physical position by computing how far it has drifted from the ideal center and applying corrective steering to maintain its trajectory. In doing so, the robot uses two color sensors on each side of its front which adjust it from a disturbed state to an ideal state.

The Ideal State, described in the code by the variable `setpoint`, is the state in which the robot is perfectly centered, and both sensors read the exact reflection value from the white, meaning the difference is 0.

The Disturbed State is the state where the robot, by drifting left or right, moves over the black boundary line. This causes the sensor’s value on the black line to drop while the other sensor remains on white, so it keeps its value. In this situation, the difference between the two sensors becomes non-zero and it needs to be fed to control algorithms to be corrected.

The task configuration defines the following thresholds:

Listing 4.1: Lane keeping task configuration.

```
1 if task == "LaneKeeping":
2     taskConfiguration={
3         "LeftColorSensor":    Port.S2,
4         "RightColorSensor":   Port.S3,
5         "BlackThreshold":     6,
6         "CrossingThreshold":   7,
7         "MinDifference":       3,
8         "BaseSpeed":           90,
9         "TurningSpeed":        70,
10        "EmergencySpeed":      65,
11        "EmergencyTurnSpeed":  100,
```

```
12         "StopConfirmCount": 1,  
13         "LogInterval": 1,  
14     }
```

The Left and Right Color Sensors are connected to the physical ports of the robot. The Black Threshold represents value 6 where the sensors are above the black line, while Crossing Threshold represents the value 7 where the sensors are very close to the black line. The Min Difference being 3 represents the tolerance zone where the sensors are above white surface, but they don't read the exact same value (values close to 0 means dark and values close to 100 means bright). This allows the robot to continue straight, ignoring shadows on the white surface. Base Speed is the forward velocity of the robot being perfectly centered. It is 50 mm/s since the track isn't long and can be easily followed. Turning Speed being 25 mm/s is the reduced forward velocity of the robot during a regulation to prevent the robot from overshooting its turns. Emergency Speed (20 mm/s) and Turn Speed (100 deg/s) are highly restrictive forward speed, respectively extreme rotational speed, used exclusively to aggressively pivot the robot away from a lane exit. The Stop Confirm Count is the number of times the robot detects the value of black on both sensors before it stops. Due to false readings caused by shadows or oversteering in narrow corridors, this variable has been introduced. After many runs of the robot when there were no more false readings, the value of times it encounters black is set to 1. The Log Interval is the number of seconds to pass to print what the robot has recorded and what action did it took. It is 1 for a more detailed log.

4.1.2 THE LANE KEEPING CONTROLLER

The Lane Keeping application's functionality is described in the LaneKeepingController class, which implements the Controller interface. In this class there are defined the methods which contribute to the desired behavior of the robot when it tries to follow the lane. It firstly initializes the attributes in the constructor, retrieves the values from the sensors, prints the header of the log, decides what action needs to be taken and finally the run method which starts the robot to move.

Listing 4.2: LaneKeepingController constructor.

```
1 def __init__(self, algorithm, configuration, streamer, ev3, robot):  
2     super().__init__(algorithm, configuration, streamer, ev3, robot)  
3     self.leftSensor=ColorSensor(configuration["LeftColorSensor"])  
4     self.rightSensor=ColorSensor(configuration["RightColorSensor"])  
5     self.blackThreshold=configuration["BlackThreshold"]  
6     self.crossingThreshold=configuration["CrossingThreshold"]  
7     self.minDifference=configuration["MinDifference"]  
8     self.baseSpeed=configuration["BaseSpeed"]  
9     self.turningSpeed=configuration["TurningSpeed"]  
10    self.emergencySpeed=configuration["EmergencySpeed"]  
11    self.emergencyTurnSpeed=configuration["EmergencyTurnSpeed"]  
12    self.stopConfirmCount=configuration["StopConfirmCount"]  
13    self.logInterval=configuration["LogInterval"]
```

The constructor takes the inherited attributes from the super abstract class Controller

and along with the more specific attributes taken from the configuration dictionary. These attributes initialize the state of the LaneKeepingController object.

Listing 4.3: LaneKeepingController sensors reading.

```
1 def readSensors(self):
2     leftValue=self.leftSensor.reflection()
3     rightValue=self.rightSensor.reflection()
4     return leftValue, rightValue
```

This method simply reads the values of the light reflected on the surface they are currently on and returns them.

Listing 4.4: LaneKeepingController header printing.

```
1 def printHeader(self, algorithm):
2     self.ev3.speaker.beep()
3     print("\nLANE KEEPING-"+algorithm+"\n")
4     print("Black Threshold:"+ str(self.blackThreshold))
5     print("Crossing Threshold:"+ str(self.crossingThreshold))
6     print("Min Difference:"+ str(self.minDifference))
7     print("Base Speed:"+ str(self.baseSpeed))
8     print("Turning Speed:"+ str(self.turningSpeed))
9     print("Emergency Speed:"+ str(self.emergencySpeed)+"\n")
```

This method prints the header of the log to be able to compare the results and see how the thresholds affect the decision-making process after the run.

Listing 4.5: LaneKeepingController action deciding.

```
1 def decideAction(self, leftValue, rightValue, difference, dt):
2     if leftValue<self.crossingThreshold and rightValue>=self.
3         crossingThreshold:
4         return self.emergencySpeed, self.emergencyTurnSpeed
5
6     if rightValue<self.crossingThreshold and leftValue>=self.
7         crossingThreshold:
8         return self.emergencySpeed, -self.emergencyTurnSpeed
9
10    if abs(difference)<=self.minDifference and isinstance(self.
11        algorithm, (PIDAlgorithm, FuzzyPIAlgorithm)):
12        return self.baseSpeed, 0
13
14    disturbance, turnRate=self.algorithm.compute(0, difference, dt)
15    self.streamer.send(self.step, disturbance, turnRate)
16
17    return self.turningSpeed, turnRate
```

This method implements a safety layered architecture with three priority levels: emergency override, admitted tolerance and algorithm regulation. Based on this architecture, the method decides what action needs to be taken based on the values read by the sensors and the difference between them.

The first case treats the moment when the left sensor reads a value close to the black and the right sensor reads a larger value. This means that the robot is crossing the black line on the left side, so the method returns the emergency speed and the emergency turn speed with positive value, meaning turning right.

The second case is opposite to the first as it treats the moment when the right sensor reads a value close to the black and the left sensor reads a larger value. This means that the robot is crossing the black line on the right side, so the method returns the emergency speed and the emergency turn speed with negative value, meaning turning left.

The third case treats the difference with the values of the sensors being smaller or equal than the minimum difference accepted to let the robot continue straight ahead, meaning that the robot is on a bright surface and the method returns the normal speed and the speed of turning is 0 to move without deviation.

In the fourth case the disturbance and turn rate are taken from the results done by the compute() method of the selected algorithm and then sent to the PC. The method finally returns the turning speed and the turn rate (the output of the regulation) meaning that the robot is regulated by the selected algorithm.

Listing 4.6: LaneKeepingController run loop.

```
1 def run(self):
2     self.printHeader(self.algorithmLabel())
3
4     bothSensorsOnLineCount=0
5     lastTime=0.0
6
7     try:
8         while True:
9             step, currentTime=self.nextStep()
10            dt=currentTime-lastTime
11            lastTime=currentTime
12
13            leftValue, rightValue=self.readSensors()
14
15            if step%self.logInterval==0:
16                print("Step "+str(step)+" Left: "+str(leftValue)+"
17                    Right: "+str(rightValue))
18                if isinstance(self.algorithm, FuzzyRuleBased):
19                    print(self.algorithm.logRules())
20
21            leftOnBlack=leftValue<self.blackThreshold
22            rightOnBlack=rightValue<self.blackThreshold
23
24            if leftOnBlack and rightOnBlack:
25                bothSensorsOnLineCount+=1
26                if bothSensorsOnLineCount>=self.stopConfirmCount:
27                    self.robot.stop()
28                    self.ev3.speaker.beep()
29                    print("\nBoth sensors on line for "+str(self.
```

```

        stopConfirmCount)+" consecutive steps.
        Stopping.")
    break

    self.robot.drive(10, 0)
    wait(20)
    continue
else:
    bothSensorsOnLineCount=0

    difference=leftValue-rightValue
    speed, turnRate=self.decideAction(leftValue, rightValue,
        difference, dt)

    if speed<10:
        speed=10

    self.robot.drive(speed, turnRate)
    wait(20)

except KeyboardInterrupt:
    self.robot.stop()
    self.ev3.speaker.beep()
    print("\nLANE KEEPING STOPPED\n")
finally:
    self.streamer.close()
```

The final run() method is the main execution loop. It combines four phases of the control cycle: sense, decide, actuate and wait. It includes the previous three methods to execute the task. Firstly, it identifies the algorithm by checking the instance of the algorithm class and attributes it to the algorithm variable. It prints the header according to the algorithm and the constants and resets the number of times both sensors crossed the black line and the time.

The continuous loop begins with the computation of the change in time by subtracting the current time (obtained from the now() method) with the last time and updating the last time. The left and right values are assigned with the values read by the sensors and then, based on the log interval, which in this case is 1, is printed at each multiple of it the step plus what values the sensors read. If the algorithm is Absolute Fuzzy, the rules are also printed.

With the help of the Boolean variables left on black and right on black, the condition for the robot to stop is created by having both values of the sensors smaller than the threshold value which indicates a very dark surface. In the event of false readings, the number of times both sensors were on the black line increases until it reaches the confirmation count to stop, otherwise it reduces the speed and waits to see if indeed is on a black line or not.

If the conditions fail, the difference is computed and the speed with the turn rate are computed from the action it needs to be decided based on the left value, right value, difference and the change in time. The returned speed can't get lower than 10 and the robot is driving based on the final speed and turn rate. The final wait is used to ensure that the loop frequency remains consistent regardless of which branch was taken.

The except part of the try-except block where the program is interrupted by the user. The robot stops and the stream closes.

4.1.3 PID ALGORITHM

One possible option for the robot to succeed in following the lane is with the help of a PID controller. This algorithm that implements this type of controller uses Comparative Control, where the disturbance of the two color sensors, is computed at each step in order to be given as input to the controller so that it can generate the turn rate so that the disturbance is minimized.

Listing 4.7: PIDAlgorithm constructor.

```
1 def __init__(self, configuration):
2     self.kp=configuration["Kp"]
3     self.ki=configuration["Ki"]
4     self.kd=configuration["Kd"]
5     self.minOutput=configuration["MinOutput"]
6     self.maxOutput=configuration["MaxOutput"]
7     self.integralMin=configuration["IntegralMin"]
8     self.integralMax=configuration["IntegralMax"]
9     self.integral=0.0
10    self.previousDisturbance=0.0
```

In the first part of the definition of the class, the constructor initializes the constants which will be computed to determine the final turn rate, as well as the interval ends for it and the integral. The previous disturbance and the integral are set to 0.

Listing 4.8: PIDAlgorithm compute method.

```
1 def compute(self, setPoint, measuredPoint, dt):
2     disturbance=setPoint-measuredPoint
3     pTerm=self.kp*disturbance
4
5     if dt>0:
6         self.integral+=disturbance*dt
7
8         if self.integral>self.integralMax:
9             self.integral=self.integralMax
10        elif self.integral<self.integralMin:
11            self.integral=self.integralMin
12        iTerm=self.ki*self.integral
13
14    if dt>0:
15        derivative=(disturbance-self.previousDisturbance)/dt
16    else:
17        derivative=0.0
18
19    dTerm=self.kd*derivative
20
```

```
21         output=pTerm+iTerm+dTerm
22
23         if output>self.maxOutput:
24             output=self.maxOutput
25         elif output<self.minOutput:
26             output=self.minOutput
27
28         self.previousDisturbance=disturbance
29
30         return disturbance, output
```

The `compute()` method takes the `setPoint`, which is usually 0 indicating perfect balance between sensors, the `measuredPoint` which is the actual difference between sensors, and the change in time. The disturbance is the result of ideal minus actual. The final turn rate output is the sum of the three PID terms made of the imbalance and their gains:

$$\text{Turn Rate} = (K_P \times \text{Imbalance}) + \left(K_I \times \int \text{Imbalance} dt \right) + \left(K_D \times \frac{d(\text{Imbalance})}{dt} \right) \quad (4.1)$$

The Derivative Gain symbolizes the present as it only considers the current value of the Disturbance. As the main steering force, its effect on the robot is that the further the robot is from the center, the harder it turns. In other words, if the robot is sluggish and goes off the track on curves, increase the K_P . If the robot shakes violently back and forth, decrease the K_P . In the `compute()` method the proportional term is the product of the disturbance and the gain.

The Integral Gain represents the memory since it accumulates the Disturbance over time. If the robot has a mechanical drift, for example a slightly weaker motor, or is on a long, sweeping curve, K_P alone might leave the robot off-center while K_I builds slowly steering power until the robot is perfectly centered. Usually this constant is very small, less than 0.1. Otherwise, it creates Integral Windup, meaning that the robot will overreact drastically and swing off the lane. In the `compute()` method the integral term is the product of gain and the disturbance multiplied with the change in time, constrained by the integral interval in order to prevent the integral from growing infinitely large.

The Derivative Gain is the dampener as it computes how fast the Disturbance is changing. When the robot steers back toward the center of the lane, K_D acts as a brake. It reduces the steering power right before the robot reaches the center, preventing it from overshooting into the other black line. A higher K_D will tighten the path so that the robot will avoid crawling widely as a snake down the lane. In the `compute()` method the derivative term is the product of the gain and the derivative of the disturbance with respect to the time.

The output final turn rate is comprised of all three PID terms and restricted to the output interval so that the motors of the robot don't receive impossible commands.

Listing 4.9: PIDAlgorithm reset method.

```
1 def reset(self):
2     self.integral=0.0
3     self.previousDisturbance=0.0
```

The previous disturbance is updated with the current one and the method returns the disturbance and the output. The PIDAlgorithm class also has a reset method to clear the memory of the controller by setting the integral and the previous disturbance to 0.

4.1.4 FUZZY PI ALGORITHM

This algorithm stands apart from a rigid mathematical model as the PID controller by acting as a human driver who adjusts the steering power based on how off the center the robot is. A Fuzzy PI controller changes the constants K_P and K_I depending on the state of the robot using Fuzzy Logic.

In standard programming the logic is binary, where conditions are either true (1) or false (0). Fuzzy Logic introduces degrees of truth, allowing the robot to categorize a disturbance slightly off-center or very far off-center. By translating the crisp value of the Disturbance into fuzzy sets, the algorithm adjusts dynamically the proportional and integral gains to generate the final turn rate.

Listing 4.10: FuzzyPIAlgorithm constructor.

```
1 def __init__(self, configuration):
2     self.baseKp=configuration["BaseKP"]
3     self.baseKi=configuration["BaseKI"]
4     self.smallDisturbance=configuration["SmallDisturbance"]
5     self.mediumDisturbance=configuration["MediumDisturbance"]
6     self.largeDisturbance=configuration["LargeDisturbance"]
7     self.fuzzyMinOutput=configuration["FuzzyMinOutput"]
8     self.fuzzyMaxOutput=configuration["FuzzyMaxOutput"]
9     self.fuzzyIntegralMin=configuration["FuzzyIntegralMin"]
10    self.fuzzyIntegralMax=configuration["FuzzyIntegralMax"]
11    self.rules=configuration["Rules"]
12    self.integral=0.0
```

The FuzzyPIAlgorithm class implements the fuzzy PI algorithm following the three stages of a fuzzy control system: fuzzification, fuzzy inference and defuzzification. The fuzzification is the conversion of a crisp numerical value into a fuzzy set. The fuzzy inference is the approximate reasoning to formulate a mapping from premise to conclusion. The defuzzification is the reverse conversion of a fuzzy set into a crisp value. The constructor of the fuzzy PI algorithm initializes the base gains K_P and K_I which will be computed to determine the final turn rate, the fuzzy set boundaries $S=3$, $M=6$, $L=10$ describing the disturbance, the rule tuples that encode the gain multipliers for each fuzzy set.

Fuzzification

Listing 4.11: FuzzyPIAlgorithm fuzzification.

```
1 def fuzzifyDisturbance(self, disturbance):
2     disturbanceAbs=abs(disturbance)
3
```

```
4         if disturbanceAbs<=self.smallDisturbance:
5             muSmall=1.0
6         elif disturbanceAbs<=self.mediumDisturbance:
7             muSmall=(self.mediumDisturbance-disturbanceAbs)/(self.
8                 mediumDisturbance-self.smallDisturbance)
9         else:
10             muSmall=0.0
11
12         if disturbanceAbs<=self.smallDisturbance:
13             muMedium=0.0
14         elif disturbanceAbs<=self.mediumDisturbance:
15             muMedium=(disturbanceAbs-self.smallDisturbance)/(self.
16                 mediumDisturbance-self.smallDisturbance)
17         elif disturbanceAbs<=self.largeDisturbance:
18             muMedium=(self.largeDisturbance-disturbanceAbs)/(self.
19                 largeDisturbance-self.mediumDisturbance)
20         else:
21             muMedium=0.0
22
23         if disturbanceAbs<=self.mediumDisturbance:
24             muLarge=0.0
25         elif disturbanceAbs<=self.largeDisturbance:
26             muLarge=(disturbanceAbs-self.mediumDisturbance)/(self.
27                 largeDisturbance-self.mediumDisturbance)
28         else:
29             muLarge=1.0
30
31         return muSmall, muMedium, muLarge
```

The `fuzzifyDisturbance()` method takes the crisp value of the disturbance and computes the membership degree for each fuzzy set: small S , medium M and large L . The μ_S , μ_M , μ_L , representing the membership degrees, are values between 0 and 1 and describe how much the input resembles each set simultaneously.

The three fuzzy sets are defined by triangular and trapezoidal membership functions. They are delimited by the boundary values from the configuration: $S=3.0$, $M=6.0$ and $L=10.0$. These boundaries determine where one fuzzy set begins and another ends, with overlapping regions where the disturbance partially belongs to two sets at the same time. It checks for each fuzzy set because there can be more than one non-zero membership value, and that's why the method returns all three membership values.

The picture above presents graphically the intervals where the disturbance can be situated and what membership value can have, describing a triangular/trapezoidal transition. The disturbance grows or is reduced resulting in smaller membership function for the initial set and a higher membership function for the next set.

For *Small*, a trapezoidal membership function, has full membership ($\mu_S = 1.0$) when $|d| \leq S$. It then decreases linearly from 1 to 0 as the disturbance increases from S to M . Zero membership ($\mu_S = 0.0$) when $|d| \geq M$.

For *Medium*, a triangular membership function, has zero membership ($\mu_M = 0.0$) when $|d| \leq S$ or $|d| \geq L$. It increases linearly from 0 to 1 as the disturbance increases from S

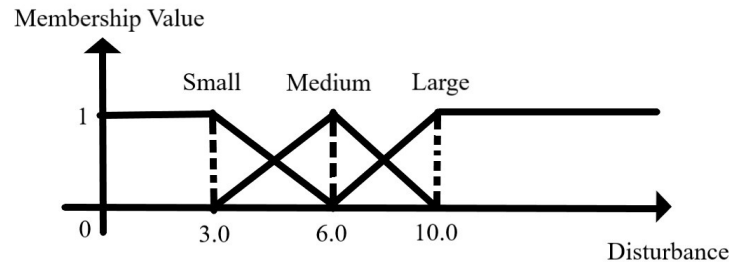


Figure 4.1: Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.

to M until it reaches its peak when the disturbance is exactly M and has full membership function ($\mu_M = 1.0$). Then it decreases linearly from 1 to 0 as the disturbance increases from M to L .

For *Large*, a trapezoidal membership function, has zero membership ($\mu_L = 0.0$) when $|d| \leq M$. Then it increases linearly from 0 to 1 as the disturbance increases from M to L and reaches full membership ($\mu_L = 1.0$) for any disturbance greater or equal to L .

These membership functions have the property that at any value of the disturbance the sum of membership values is 1. This ensures a smooth transition between the fuzzy sets and better control for the robot's movement.

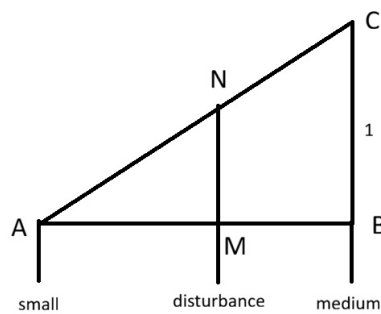


Figure 4.2: Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.

This picture represents the ascending slope of the medium membership function and the disturbance having a value between small and large. It creates two similar right triangles, ABC which is formed by the full slope and AMN which is the smaller triangle formed by the

membership value of the disturbance.

This means that MN/BC is equal to AM/AB , so the side $MN = AM \times BC/AB$. Since BC is equal to 1, AM is disturbance – small and AB is medium – small, the membership degree of the disturbance (MN) is (disturbance – small)/(medium – small). This geometric reasoning is applied to every ascending and descending slope for each membership function in order to compute every membership value.

Fuzzy Inference Rules & Defuzzification

Listing 4.12: FuzzyPIAlgorithm inference & defuzzification.

```

1 def inference(self, muSmall, muMedium, muLarge):
2     kpSmall = self.rules[0][0]
3     kiSmall = self.rules[0][1]
4
5     kpMedium = self.rules[1][0]
6     kiMedium = self.rules[1][1]
7
8     kpLarge = self.rules[2][0]
9     kiLarge = self.rules[2][1]
10
11    totalWeight = muSmall + muMedium + muLarge
12
13    if totalWeight > 0:
14        kpMultiplier = (muSmall * kpSmall + muMedium * kpMedium +
15                        muLarge * kpLarge) / totalWeight
16        kiMultiplier = (muSmall * kiSmall + muMedium * kiMedium +
17                        muLarge * kiLarge) / totalWeight
18    else:
19        kpMultiplier = 1.0
20        kiMultiplier = 1.0
21
22    kpAdaptive = self.baseKp * kpMultiplier
23    kiAdaptive = self.baseKi * kiMultiplier
24
25    return kpAdaptive, kiAdaptive

```

The inference stage applies a set of IF-THEN rules that map each fuzzy set to specific gain multiplier values. These rules encode the control strategy: how aggressively the controller should respond to the magnitude of the disturbance. The `inference()` method handles the inference and defuzzification stages of the fuzzy control system. The Fuzzy PI algorithm uses three rules, stored in the configuration (the list Rules: $[(0.8, 0.7), (1.0, 1.0), (1.2, 1.3)]$) where each tuple represents the K_P and K_I multiplier for the corresponding fuzzy set.

Rule 1: IF the disturbance is small THEN $K_P \times 0.8$ and $K_I \times 0.7$. This is interpreted as: If the disturbance between the values read by the sensors is small and the robot is near the center then the gains are reduced, K_P by 20% and K_I by 30%, for smooth and gentle corrections. This prevents the robot from making oscillations around the center position.

Rule 2: IF the disturbance is medium THEN $K_P \times 1.0$ and $K_I \times 1.0$. This is interpreted as: If the disturbance is medium and the robot is moderately off-center then keep the nominal gains. This is the nominal point of the controller.

Rule 3: IF the disturbance is large THEN $K_P \times 1.2$ and $K_I \times 1.3$. This is interpreted as: If the disturbance is large and the robot is far off-center then boost the gains, K_P by 20

The defuzzification stage is the conversion of the fuzzy set back into a crisp value. The method mentioned previously computes a weighted average defuzzification with the membership values serving as weights. From the inference stage, the controller combines the gain multipliers by aggregating all active rules into a pair of final adaptive gains. The denominator, being the sum of all membership values, is always equal to 1 and ensures that the result is normalized. These adaptive gains are then given to the `compute()` method to obtain the final turn rate.

Computing the Output

Listing 4.13: FuzzyPIAlgorithm output computing

```
1 def compute(self, setPoint, measuredPoint, dt):
2     disturbance=setPoint-measuredPoint
3     muSmall, muMedium, muLarge=self.fuzzifyDisturbance(disturbance)
4     kpAdaptive, kiAdaptive=self.inference(muSmall, muMedium, muLarge)
5
6     pTerm=kpAdaptive*disturbance
7
8     if dt>0:
9         self.integral+=disturbance*dt
10
11         if self.integral>self.fuzzyIntegralMax:
12             self.integral=self.fuzzyIntegralMax
13         elif self.integral<self.fuzzyIntegralMin:
14             self.integral=self.fuzzyIntegralMin
15
16     iTerm=kiAdaptive*self.integral
17
18     output=pTerm+iTerm
19
20     if output>self.fuzzyMaxOutput:
21         output=self.fuzzyMaxOutput
22     elif output<self.fuzzyMinOutput:
23         output=self.fuzzyMinOutput
24
25     return disturbance, output
```

The `compute()` method combines all stages of the fuzzy logic to obtain the final turn rate the robot must execute. It receives the `setPoint` which is usually 0, indicating perfect balance between sensors, the `measuredPoint` which is the actual difference between sensors and the change in time. The disturbance is the difference between `setPoint` and

measuredPoint. Using the adaptive K_P and K_I computed from the fuzzy system, there are constructed the PI terms of the controller.

The proportional term is the product of the disturbance and the adaptive gain and acts as the correction to the current disturbance, while the integral term is the product of gain and the disturbance multiplied with the change in time, constrained by the integral interval in order to prevent the integral from growing infinitely large.

$$Output = (K_P \times Imbalance) + (K_I \times \int Imbalance dt) \quad (4.2)$$

The output final turn rate is comprised of the two PI terms and restricted to the output interval, so that the motors of the robot don't receive impossible commands. The method returns the disturbance and the output representing the turn rate.

The `reset()` method is used to clear the integral accumulator, and it's called whenever the controller is reinitialized. For example, after the robot handles a lane crossing event.

The advantage of this approach over a fixed-gain PID controller is that the Fuzzy PI controller automatically adjusts its output according to its position from the center. It weakens when the robot is close to the center to avoid jitters and strengthens the output when the robot is far from the center to avoid sluggish reaction to large deviations. In this behavior, the fuzzy logic provides a smooth transition between control strategies.

4.1.5 FUZZY RULE-BASED ALGORITHM

This algorithm relies only on fuzzy logic to control the movement of the robot based on disturbance created by the difference between values read from the sensors. Unlike the Fuzzy PI algorithm, which only tunes the gains of the PI controller, the value of disturbance and the derivative of disturbance are fuzzified and mapped through a complete Mamdani type fuzzy inference system directly to the turn rate given to the motor.

This approach has two advantages. The first one is the control strategy is expressed in natural language rules rather than mathematical gain values. This makes it easier to tune the boundaries of the fuzzy sets. The second one is by using the current disturbance and the rate of change (trend) the controller is able to anticipate future states. The correction is weakened when the robot is recovering to the center and it is strengthened when the robot begins to deviate from the center.

The algorithm is implemented in the `FuzzyRuleBased` class and comprises of the four stages of the Mamdani fuzzy controller: fuzzification of the disturbance and the trend, fuzzy inference through a set of 15 rules, aggregation of the active conclusion rules and defuzzification to obtain the crisp final turn rate.

Listing 4.14: `FuzzyRuleBasedAlgorithm` constructor

```
1 class FuzzyRuleBased(Algorithm):
2     def __init__(self, configuration):
3         self.smallDisturbance=configuration["SmallDisturbance"]
4         self.mediumDisturbance=configuration["MediumDisturbance"]
```

```
5         self.deltaThreshold=configuration["DeltaThreshold"]
6         self.smallOutput=configuration["SmallOutput"]
7         self.mediumOutput=configuration["MediumOutput"]
8         self.largeOutput=configuration["LargeOutput"]
9         self.rulesTable=configuration["Rules"]
10
11        self.previousDisturbance=0.0
12        self.firstCall = True
13        self.lastActiveRules=[]
14
15        self.outputSingletons=[
16            -self.largeOutput,
17            -self.mediumOutput,
18            -self.smallOutput,
19            0.0,
20            self.smallOutput,
21            self.mediumOutput,
22            self.largeOutput,
23        ]
```

The FuzzyRuleBased begins with the constructor, initializing from the configuration the fuzzy sets for disturbance, derivative of disturbance and the output, the interval ends restricting the output and the fuzzy rules. The interval ends are necessary to prevent the motor of the robot from executing impossible commands. The previous disturbance is set to 0 to compute the derivative. The variables first call, last active rules and previous active rules keep track of which rules are firing and by how much to debug easier. The output singletons define the magnitude of the turn rate.

Listing 4.15: FuzzyRuleBasedAlgorithm clamping

```
1 def clamp(self, value):
2     if value<0:
3         return 0.0
4     elif value>1:
5         return 1.0
6     return value
```

The clamp method is a helper function which limits the range of the membership value of each fuzzy set to be between 0 and 1.

Fuzzification of Disturbance

Listing 4.16: FuzzyRuleBasedAlgorithm disturbance fuzzification

```
1 def fuzzifyDisturbance(self, disturbance):
2     S=self.smallDisturbance
3     M=self.mediumDisturbance
4
5     muNL=self.clamp((-S-disturbance)/(M-S))
```

```

6
7     if disturbance<=-M or disturbance>=0.0:
8         muNS=0.0
9     elif disturbance<=-S:
10        muNS=(disturbance+M) / (M-S)
11    else:
12        muNS=(-disturbance) / S
13
14    if disturbance<=-S or disturbance>=S:
15        muZE=0.0
16    elif disturbance<0.0:
17        muZE=(disturbance+S) / S
18    else:
19        muZE=(S-disturbance) / S
20
21    if disturbance<=0.0 or disturbance>=M:
22        muPS=0.0
23    elif disturbance<=S:
24        muPS=disturbance/S
25    else:
26        muPS=(M-disturbance) / (M-S)
27
28    muPL=self.clamp((disturbance-S) / (M-S))
29
30    return (muNL, muNS, muZE, muPS, muPL)

```

The `fuzzifyDisturbance()` method takes the crisp value of the disturbance, as the first input, and computes the membership degree for each fuzzy set: negative large NL, negative small NS, zero ZE, positive small PS and positive large PL. The μ_{NL} , μ_{NS} , μ_{ZE} , μ_{PS} and μ_{PL} represents the membership degrees, with values between 0 and 1 and describe how much the input resembles each set simultaneously. The reason these fuzzy sets are defined on signed disturbances, unlike the Fuzzy PI algorithm, is that this allows the controller to distinguish between left and right deviation.

These five fuzzy sets are defined by triangular and trapezoidal membership functions. They are delimited by the boundary and signed values from the configuration: $-S=-3.0$, $-M=-6.0$, $+S=3.0$, $+M=6.0$. These boundaries determine where one fuzzy set begins and another ends, with overlapping regions where the disturbance partially belongs to two sets at the same time. It checks for each fuzzy set because there can be more than one non-zero membership value, and that's why the method returns all five membership values.

The picture above presents graphically the intervals where the disturbance can be situated and what membership value can have, describing a triangular/trapezoidal transition. The disturbance grows or is reduced resulting in smaller membership function for the initial set and a higher membership function for the next set.

For negative large, a trapezoidal membership function, has full membership ($\mu_{NL} = 1.0$) when $d \leq -M$. Then it decreases linearly from 1 to 0 as the disturbance increases from $-M$ to $-S$. Zero membership ($\mu_{NL} = 0.0$) when $d > -S$. This represents a strong drift to the left.

For negative small, a triangular membership function, has zero membership ($\mu_{NS} = 0.0$) when $d < -M$ or $d > 0$. It increases linearly from 0 to 1 as the disturbance increases from

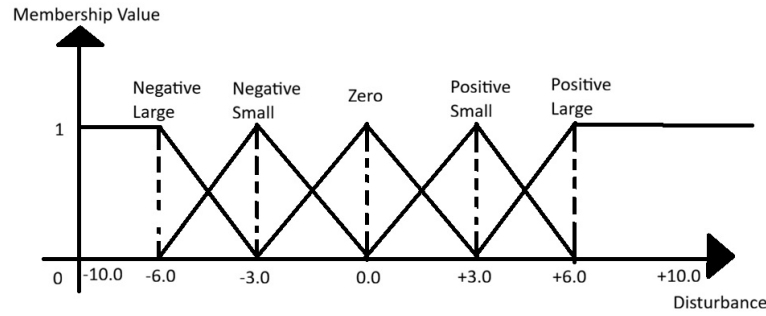


Figure 4.3: Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.

-M to -S until it reaches its peak when the disturbance is exactly -S and has full membership ($\mu_{NS} = 1.0$). Then it decreases linearly from 1 to 0 as the disturbance increases from -S to 0. This represents a slight drift to the left.

For zero, a triangular membership function, has zero membership ($\mu_{ZE} = 0.0$) when $d < -S$ or $d > +S$. It increases linearly from 0 to 1 as the disturbance increases from -S to 0 until it reaches its peak when the disturbance is exactly 0 and has full membership ($\mu_{ZE} = 1.0$). Then it decreases linearly from 1 to 0 as the disturbance increases from 0 to +S. This represents the robot being approximately centered in the lane.

For positive small, a triangular membership function, has zero membership ($\mu_{PS} = 0.0$) when $d < 0$ or $d > +M$. It increases linearly from 0 to 1 as the disturbance increases from 0 to +S until it reaches its peak when the disturbance is exactly S and has full membership ($\mu_{PS} = 1.0$). Then it decreases linearly from 1 to 0 as the disturbance increases from +S to +M. This represents a slight drift to the right.

For positive large, a trapezoidal membership function, has full membership ($\mu_{PL} = 1.0$) when $d \geq M$. It decreases linearly from 1 to 0 as the disturbance decreases from +M to +S. Zero membership ($\mu_{PL} = 0.0$) when $d < +M$.

This picture represents the ascending slope of the medium membership function and the disturbance having a value between small and large. It creates two similar right triangles, ABC which is formed by the full slope and AMN which is the smaller triangle formed by the membership value of the disturbance.

This means that MN/BC is equal to AM/AB , so the side $MN = AM \times BC/AB$. Since BC is equal to 1, AM is *disturbance - small* and AB is *medium - small*, the membership degree of the disturbance (MN) is $(disturbance - small)/(medium - small)$. This geometric reasoning is applied to every ascending and descending slope for each membership function in order to compute every membership value, as was proved for the Fuzzy PI algorithm. The trapezoidal functions have only one slope and the membership values are implemented in the

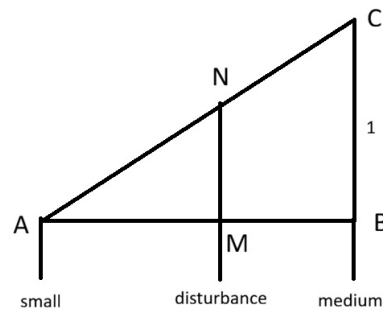


Figure 4.4: Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.

code with the help from the `clamp()` method to restrict the results to the interval $[0, 1]$, while the triangular functions have two mirrored slopes.

Fuzzification of Trend

The second input for the fuzzy control system is how the disturbance is changing between consecutive control cycles. It is computed as $\Delta d = d_{\text{current}} - d_{\text{previous}}$. The physical interpretation of the trend depends on its sign. A positive trend means the new disturbance is higher than the previous one, so the robot is moving further from the center while a negative trend means the new disturbance is lower than the previous one, so the robot is recovering towards the center.

Listing 4.17: FuzzyRuleBasedAlgorithm trend fuzzification

```

1 def fuzzifyDelta(self, delta):
2     D=self.deltaThreshold
3
4     muN=self.clamp((-delta)/D)
5
6     if delta<=-D or delta>=D:
7         muZ=0.0
8     elif delta<0.0:
9         muZ=(delta+D)/D
10    else:
11        muZ=(D-delta)/D
12
13    muP=self.clamp(delta/D)
14
15    return (muN, muZ, muP)

```

The `fuzzifyDelta()` method takes the crisp value of trend of disturbance. It computes the membership degree for each fuzzy set: negative N, zero Z, and positive P. The μ_N , μ_Z and μ_P represents the membership degrees, with values between 0 and 1 and describe how much the input resembles each set simultaneously.

These three fuzzy sets are defined by triangular and trapezoidal membership functions. They are delimited by the threshold value from the configuration: $D=3.0$.

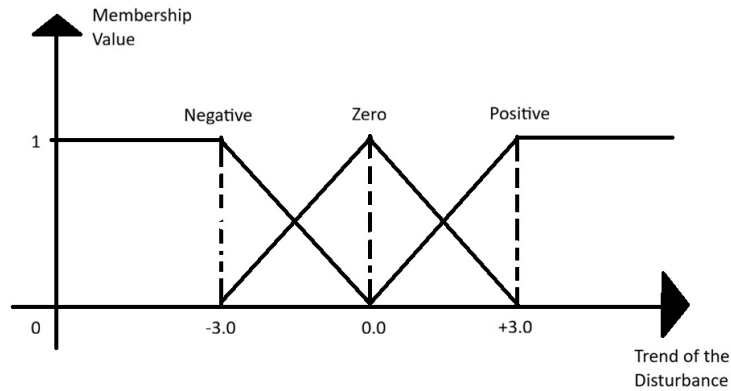


Figure 4.5: Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.

The picture above presents graphically the intervals where the trend of disturbance can be situated and what membership value can have, describing a triangular/trapezoidal transition.

For negative, a trapezoidal function, has full membership when $\Delta d \leq -D$ and decreases linearly to 0 at $\Delta d = 0$. This represents the fact that the disturbance tends to decline and the robot recovers to the center.

For zero, a triangular function, has full membership at the peak $\Delta d = 0$ and decreases linearly to the base from $-D$ to $+D$. This represents a stable disturbance with little change.

For positive, a trapezoidal function, has full membership when $\Delta d \geq +D$ and decreases linearly to 0 at $\Delta d = 0$. This represents the fact that the disturbance tends to grow and the robot furthers away from the center.

Fuzzy Inference Rules

The core of the Fuzzy Rule-Based controller is a set of 15 IF-THEN rules (five fuzzy sets for the disturbance and three for the trend) that map every combination of disturbance and trend to a final output turn rate. The output is expressed as seven fuzzy sets (negative large NL = -50, negative medium NM, negative NS, zero ZE, positive small PS, positive medium PM and positive large PL). Each set is associated with a singleton value that represents the magnitude of turn rate in degrees per second in the table below:

Table 4.1: Fuzzy Rule-Based: output labels and their crisp turn rate values.

NL	NM	NS	ZE	PS	PM	PL
-50	-30	-10	0	+10	+30	+50

Negative output values command the robot to turn left, positive values command the robot to turn right and zero commands the robot to continue straight. The singleton values were chosen to express the range of the EV3 DriveBase, from gentle to aggressive corrections, while remaining below the emergency turn speed of 100 deg/s.

The 15 rules are put in a 5×3 matrix, whose rows correspond to the five disturbance fuzzy sets and columns to the three trend of disturbance fuzzy sets:

Table 4.2: Fuzzy Rule-Based: rule matrix for lane keeping.

Disturbance \ Trend	N	Z	P
NL	PM	PL	PL
NS	PS	PM	PM
ZE	ZE	ZE	ZE
PS	NS	NM	NM
PL	NM	NL	NL

Listing 4.18: FuzzyRuleBasedAlgorithm rules tuple

```

1  "Rules": [
2      (0, 0, 5), # R1: NL & N -> PM
3      (0, 1, 6), # R2: NL & Z -> PL
4      (0, 2, 6), # R3: NL & P -> PL
5      (1, 0, 4), # R4: NS & N -> PS
6      (1, 1, 5), # R5: NS & Z -> PM
7      (1, 2, 5), # R6: NS & P -> PM
8      (2, 0, 3), # R7: ZE & N -> ZE
9      (2, 1, 3), # R8: ZE & Z -> ZE
10     (2, 2, 3), # R9: ZE & P -> ZE
11     (3, 0, 2), # R10: PS & N -> NS
12     (3, 1, 1), # R11: PS & Z -> NM
13     (3, 2, 1), # R12: PS & P -> NM
14     (4, 0, 1), # R13: PL & N -> NM
15     (4, 1, 0), # R14: PL & Z -> NL
16     (4, 2, 0), # R15: PL & P -> NL
17 ]

```

The expanded rule table, encoded in the configuration as a list of tuples (disturbanceIndex, deltaIndex, outputIndex) and loaded into memory at the constructor express the IF-THEN rules and in the table below they are presented along with their physical meaning:

Table 4.3: Fuzzy Rule-Based: full IF-THEN rule set with physical meaning.

Rule	IF disturbance is	AND trend is	THEN turn rate is	Meaning
R1	NL	N	PM	Far left & recovering ⇒ moderate right turn
R2	NL	Z	PL	Far left & stable ⇒ strong right turn
R3	NL	P	PL	Far left & worsening ⇒ strong right turn
R4	NS	N	PS	Slightly left & recovering ⇒ gentle right turn
R5	NS	Z	PM	Slightly left & stable ⇒ moderate right turn
R6	NS	P	PM	Slightly left & worsening ⇒ moderate right turn
R7	ZE	N	ZE	Centered & recovering ⇒ go straight
R8	ZE	Z	ZE	Centered & stable ⇒ go straight
R9	ZE	P	ZE	Centered & worsening ⇒ go straight
R10	PS	N	NS	Slightly right & recovering ⇒ gentle left turn
R11	PS	Z	NM	Slightly right & stable ⇒ moderate left turn
R12	PS	P	NM	Slightly right & worsening ⇒ moderate left turn
R13	PL	N	NM	Far right & recovering ⇒ moderate left turn
R14	PL	Z	NL	Far right & stable ⇒ strong left turn
R15	PL	P	NL	Far right & worsening ⇒ strong left turn

The rule table has an antisymmetric structure in the case where the disturbance is stable: the output for (NL, Z) is PL (+50), while the output for (PL, Z) is NM (-50). It is equal in magnitude but opposite in sign. Similarly, (NS, Z) maps to PM (+30) and (PS, Z) maps to NM (-30). This symmetry ensures that the robot responds the same to left and right deviations.

The trend dictates the correction strength based on the situation being improved or worsened. When the trend is N (recovering) the magnitude of the correction is reduced and when the trend is P (worsening) the magnitude of the correction is increased. If the trend is Z, meaning the robot isn't recovering yet, then the magnitude of the correction remains increased.

The centered region, when the disturbance is ZE, maps every trend to ZE keeping the robot going straight regardless of previous changes in the disturbance.

Inference and Aggregation

During each control cycle, the fuzzification stage produces up to two non-zero membership values for the disturbance from adjacent sets and up to two non-zero values for the trend. This means that at any moment, 1, 2 or 4 rules can be simultaneously active, depending on whether both inputs are exactly on a set peak (1 rule), on a transition between two sets for one input and the other is on a set peak (2 rules) or both inputs are on transitions (4 rules).

Listing 4.19: FuzzyRuleBasedAlgorithm inference & aggregation

```
1 def inference(self, muDisturbance, muDelta):
2     aggregation=[0.0]*len(self.outputSingletons)
3
4     for disturbanceIndex, deltaIndex, outputIndex in self.rulesTable
5         :
6         ruleStrength=min(muDisturbance[disturbanceIndex], muDelta[
7             deltaIndex])
8
9         if ruleStrength>0.0:
10             self.lastActiveRules.append((disturbanceIndex,
11                 deltaIndex, outputIndex, ruleStrength))
12
13             if ruleStrength>aggregation[outputIndex]:
14                 aggregation[outputIndex]=ruleStrength
15
16     return aggregation
```

The inference stage, implemented in the `inference()` method, evaluates each of the 15 rules by computing the degree activation of the rule. The degree activation is computed by the fuzzy MIN operator, being the minimum of the two input membership values.

If one input has zero membership for a rule's premise, the rule doesn't activate and it's skipped. The method keeps track of the active rules and their strength in the `lastActiveRules` list for debugging.

In the case of multiple active rules map to the same singleton output, their strength must be combined. The aggregation stage uses the fuzzy MAX operator in the way that for

each output singleton, only the highest activation degree among all rules that activate it is kept. This is implemented as an array of the size corresponding to the number of singleton outputs and each entry holds the maximum degree activation for all the rules targeting that singleton.

Defuzzification

The defuzzification stage converts the aggregated fuzzy output into the crisp turn rate value. Since the output are represented as singletons, the defuzzification use weighted average:

$$\text{turn rate} = \frac{\sum_i \text{aggregation}[i] \cdot \text{singleton}[i]}{\sum_i \text{aggregation}[i]} \quad (4.3)$$

Listing 4.20: FuzzyRuleBasedAlgorithm defuzzification

```
1 def defuzzify(self, aggregation):
2     numerator=0.0
3     denominator=0.0
4
5     for i in range(len(self.outputSingletons)):
6         numerator+=aggregation[i]*self.outputSingletons[i]
7         denominator+=aggregation[i]
8
9     if denominator>0.0:
10        return numerator/denominator
11    else:
12        return 0.0
```

The sum collects each output singleton. Singletons with zero aggregated strengths don't contribute to the turn rate and if no rules are active the default output is zero meaning driving straight.

Computing the Output

Listing 4.21: FuzzyRuleBasedAlgorithm output computing

```
1 def compute(self, setPoint, measuredPoint, dt):
2     self.lastActiveRules=[]
3     disturbance=setPoint-measuredPoint
4     delta=self.updateDelta(disturbance)
5
6     muDisturbance=self.fuzzifyDisturbance(disturbance)
7     muDelta=self.fuzzifyDelta(delta)
8
9     aggregation=self.inference(muDisturbance, muDelta)
10
11    output=self.defuzzify(aggregation)
12
13    return disturbance, output
```

The `compute()` method combines all stages of the fuzzy logic to obtain the final turn rate the robot must execute. It receives the `setPoint` which is usually 0, indicating perfect balance between sensors, the `measuredPoint` which is the actual difference between sensors and the change in time.

The membership values of the disturbance and delta are retrieved from the `fuzzify()` methods. Based on them, the rules are evaluated to find out which are activated in the `inference()` method. The output turn rate is defuzzified from the aggregated fuzzy sets and is clamped so that the robot doesn't receive commands that isn't able to execute.

4.2 THE MAZE SOLVER MODULE

4.2.1 TASK DESCRIPTION

The maze solving task requires the robot to navigate through an unknown grid-based maze composed of rectangular cells (260 mm × 280 mm) separated by walls. The robot must explore every reachable cell using a Depth-First Search strategy, then return to the starting position. Between junctions the same control algorithms regulate the forward approach speed.

Table 4.4: Maze solver configuration parameters.

Parameter	Value	Description
CellDistanceNS	260 mm	North-South cell dimension.
CellDistanceEW	280 mm	East-West cell dimension.
OpenPassageThreshold	150 mm	Sensor > this ⇒ passage is open.
ObstacleThreshold	80 mm	Forward < this ⇒ hard stop.
TargetForwardDistance	150 mm	Algorithm setpoint for speed regulation.
BaseSpeed / MinSpeed	150 / 30 mm/s	Speed bounds during cell driving.
CurveSpeed / CurveTurnRate	80 mm/s / 15 deg/s	Parameters for gyro-guided 90° turns.
BackupDistance	60 mm	Reverse distance before turning.
HeadingThreshold	3°	Dead zone for gyro correction.
CenteringThreshold / Gain	20 mm / 0.1	Side-sensor corridor centering.

4.2.2 THE MAZEMAP CLASS

The `MazeMap` class maintains a coordinate-based map of the explored maze and implements the DFS navigation logic. It uses a compass-based coordinate system where the four cardinal directions are encoded as integers: 0 = North, 1 = East, 2 = South, 3 = West.

The DFS exploration follows the priority order Forward → Left → Right → Behind. The `planMove()` method iterates this priority, checking each candidate direction for two conditions: the direction must be physically open (detected by the ultrasonic sensors) and the neighbouring cell must not have been previously visited. The `planBacktrack()` method pops the path stack to return to the most recent junction with unexplored branches.

Table 4.5: Direction encoding and coordinate changes.

Direction	Index	DX	DY	Gyro Target
North	0	0	+1	0°
East	1	+1	0	90°
South	2	0	-1	180°
West	3	-1	0	270°

The navigation uses a two-phase commit pattern: `applyTurn()` updates the heading before the physical turn, then `commitMove()` pushes the current position onto the path stack, advances the coordinates, and marks the new cell as visited. When backtracking, `commitBacktrack()` pops the previous position from the stack.

4.2.3 THE MAZESOLVERCONTROLLER

The `MazeSolverController` orchestrates the physical execution of the DFS algorithm through five support routines:

Sensor Reading. The `scanForward()` method takes three consecutive readings, sorts them, and returns the median to reject ultrasonic spike noise. The `scanJunction()` method reads all three ultrasonic sensors and returns the set of open absolute directions.

Heading Alignment. After every turn, the `alignHeading()` method corrects the robot's physical orientation using the gyroscope. The error between the expected gyro target and the actual reading is normalized to $[-180^\circ, +180^\circ]$ and the robot executes a corrective point turn if the error exceeds 3° .

Corridor Centering. The `centerInCorridor()` method reads both side sensors and, when both walls are visible, computes a corrective turn proportional to the lateral offset (0.1 deg/mm, capped at $\pm 15^\circ$).

Turn Execution. The `executeTurn()` method performs gyro-guided curve turns for 90° rotations (smooth arc at 80 mm/s, 15 deg/s) and encoder-based point turns for 180° U-turns.

Cell-to-Cell Driving. The `_driveCell()` method drives one cell using the selected algorithm to regulate forward speed. The distance is tracked via trapezoidal integration: $s += \frac{1}{2}(v_{\text{prev}} + v_{\text{curr}}) \cdot \Delta t$. The loop exits when the integrated distance reaches the cell dimension or when the forward sensor detects an unexpected wall.

4.2.4 MAIN EXECUTION LOOP

The `run()` method repeats a four-phase cycle for each cell:

1. **Sense:** scan the junction with `scanJunction()`.
2. **Plan:** call `planMove()` or `planBacktrack()`.
3. **Execute:** backup → turn → align heading → centre in corridor → drive cell → commit.
4. **Repeat** until the path stack is empty (maze fully explored).

4.3 THE TELEMETRY INTERFACE

4.3.1 WIFISTREAMER

The `WifiStreamer` class provides real-time telemetry by streaming control data from the EV3 to a PC over a UDP socket. At initialization, it creates a `SOCK_DGRAM` socket and sends a metadata packet containing the task-algorithm label. During the control loop, each cycle sends a comma-separated packet with the step number, the disturbance and the algorithm output.

UDP was chosen over TCP because it is connectionless and does not block the control loop if the PC is temporarily unreachable. The trade-off is that packets may be lost, but at the 50 Hz control rate, occasional packet loss is acceptable for visualization purposes.

4.3.2 PC RECEIVER AND VISUALIZATION

On the PC side, a Python script listens on the configured port (5005) and logs incoming packets to a CSV file. The CSV data is then visualized using Matplotlib to generate the comparative plots presented in Chapter 5. For the maze solver, the streamed (x, y) cell coordinates are plotted on a grid to visualize the exploration path in real time.

The metadata packet sent at the beginning of each run identifies the task and algorithm, allowing the PC receiver to label the log files automatically (e.g. `LaneKeeping-PID.csv`, `MazeSolver-FuzzyRuleBased.csv`).

4.4 OBJECT-ORIENTED ARCHITECTURE

4.4.1 CLASS HIERARCHY

The software is organized around two parallel hierarchies connected by composition:

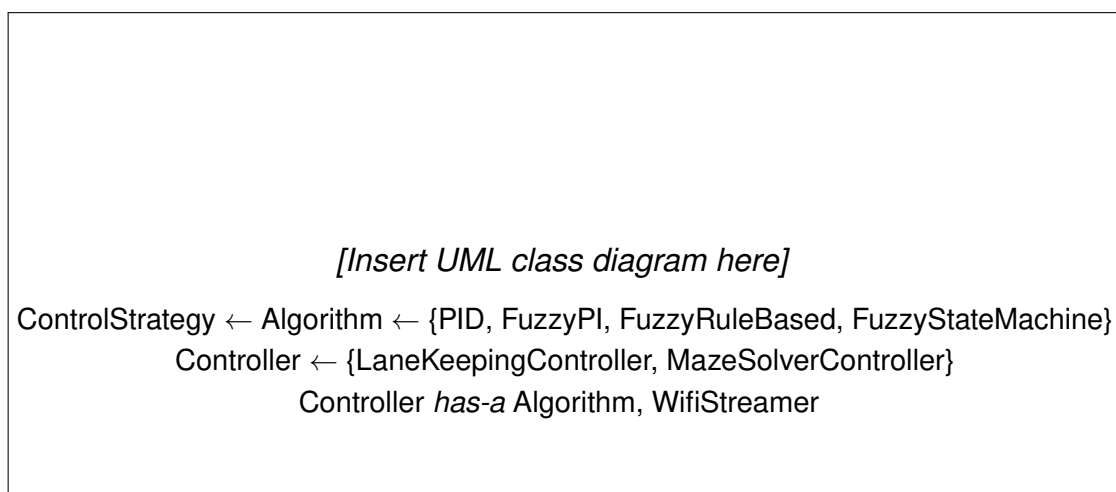


Figure 4.6: UML class diagram showing the Algorithm and Controller hierarchies.

The **Algorithm hierarchy** defines the control strategies. The abstract base

`class Algorithm` declares the `compute(setpoint, measured, dt)` interface. Three concrete subclasses implement this interface:

- `PIDAlgorithm`: fixed-gain PID controller.
- `FuzzyPIAlgorithm`: fuzzy-adaptive PI controller with 3 rules.
- `FuzzyRuleBased`: pure Mamdani controller with 15 rules and 2 inputs.

The **Controller hierarchy** defines the task-specific execution logic. The abstract base class `Controller` provides the stopwatch, step counter, and WiFi streamer. Two concrete subclasses implement the `run()` method:

- `LaneKeepingController`: continuous control loop with emergency override.
- `MazeSolverController`: DFS-based exploration with cell-to-cell driving.

4.4.2 SOLID PRINCIPLES

The architecture satisfies the five SOLID principles:

Single Responsibility: each class has one responsibility. `PIDAlgorithm` computes PID output; `MazeMap` manages graph traversal; `WifiStreamer` handles network communication.

Open/Closed: new algorithms can be added by creating a new subclass of `Algorithm` without modifying any existing code. The `FuzzyRuleBased` algorithm was added months after the initial PID implementation, requiring zero changes to the controllers.

Liskov Substitution: any `Algorithm` subclass can replace any other without breaking the controllers, because they all satisfy the same `compute()` contract.

Interface Segregation: the `Algorithm` interface exposes only `compute()` and `reset()`, rather than forcing all algorithms to implement fuzzification methods they don't need.

Dependency Inversion: the controllers depend on the abstract `Algorithm` interface. The concrete algorithm is selected at startup by the factory function `createController()`.

4.4.3 FACTORY PATTERN

The `createController(task, algorithm_name)` function serves as a factory that instantiates the correct algorithm and controller based on the configuration strings. It calls `initializeConfiguration()` to generate the parameter dictionary, `hardware_setup()` to initialize the motors, and then creates the appropriate `Algorithm` and `Controller` objects. This centralizes the creation logic and ensures that the configuration, algorithm and controller are always consistent.

5. EXPERIMENTAL RESULTS

5.1 EXPERIMENTAL SETUP

All experiments were conducted on the same physical track (for lane keeping) and the same physical maze (for maze solving) to ensure fair comparison between the three algorithms. Each algorithm was run five times, and the telemetry data was averaged. The metrics used for evaluation are:

- **Mean Absolute Disturbance (MAD):** the average of $|d|$ across all control steps, measuring how far the robot strays from the ideal state.
- **Standard Deviation of Disturbance (σ_d):** measures oscillation amplitude.
- **Crossing Count:** the number of times the emergency override was triggered.
- **Average Speed:** the mean forward velocity, reflecting how confidently the robot drives.
- **Completion Time:** total time to complete the track or maze.

5.2 LANE KEEPING RESULTS

5.2.1 DISTURBANCE OVER TIME

Figure 5.1 shows the disturbance d plotted against control steps for the three algorithms on the same track.

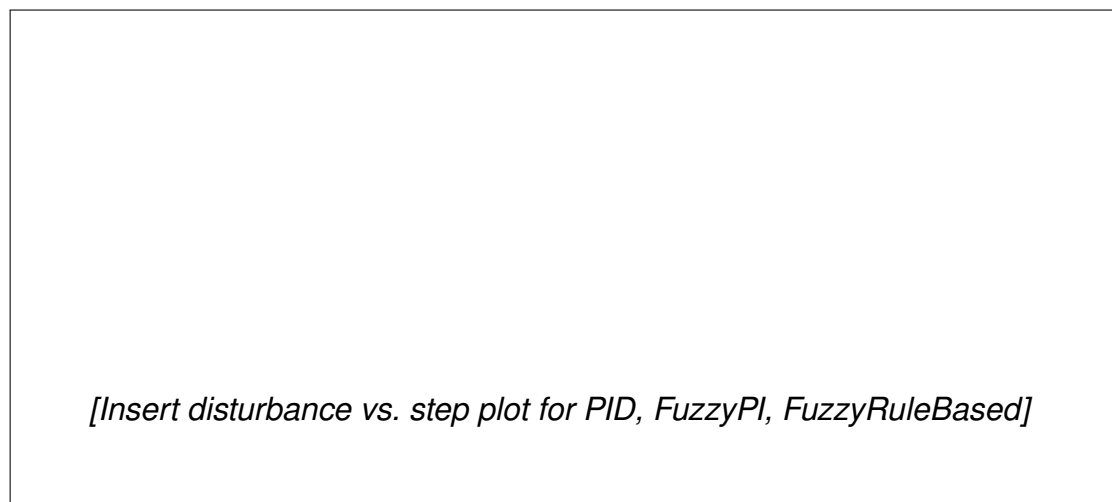


Figure 5.1: Disturbance over time for the three algorithms in the lane keeping task.

Table 5.1: Lane keeping performance comparison (average of 5 runs).

Algorithm	MAD	σ_d	Crossings	Avg Speed	Time
PID	—	—	—	— mm/s	— s
Fuzzy PI	—	—	—	— mm/s	— s
Fuzzy Rule-Based	—	—	—	— mm/s	— s

5.2.2 PERFORMANCE COMPARISON

The results show that *[fill with analysis after collecting data: which algorithm had the lowest MAD, fewest crossings, smoothest trajectory, and why the trend input in the Fuzzy Rule-Based reduces oscillations]*.

5.2.3 TURN RATE OUTPUT

Figure 5.2 compares the turn rate output of the three algorithms.

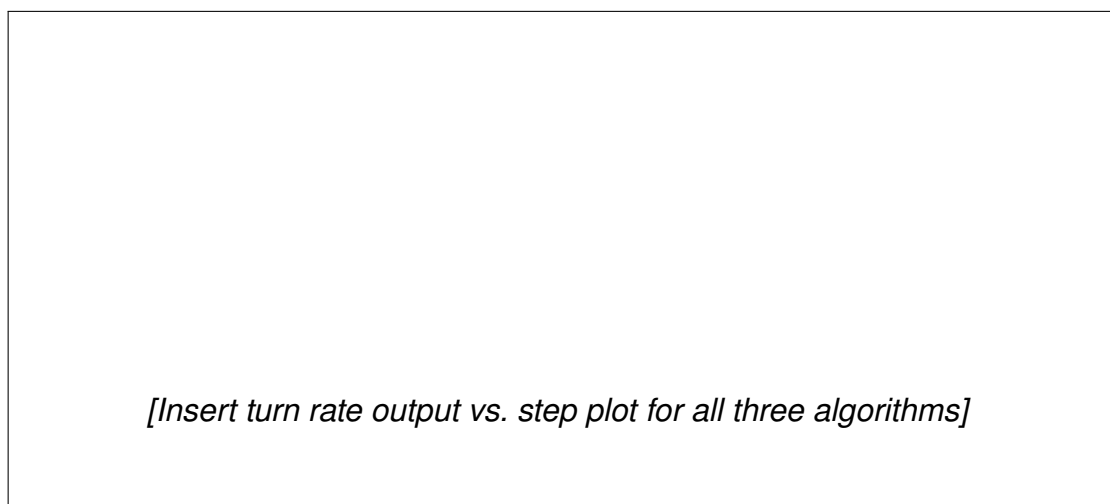


Figure 5.2: Turn rate output over time for the three algorithms.

5.2.4 RULE ACTIVATION ANALYSIS

For the Fuzzy Rule-Based algorithm, Figure 5.3 shows the distribution of rule activations across the 15 rules during a typical run.

5.3 MAZE SOLVER RESULTS

5.3.1 EXPLORATION MAP

Figure 5.4 shows the cell-by-cell exploration path reconstructed from the streamed (x, y) coordinates.

[Insert rule activation histogram or heatmap]

Figure 5.3: Rule activation frequency for the Fuzzy Rule-Based controller.

[Insert maze exploration path visualization]

Figure 5.4: Maze exploration path showing the DFS traversal order.

5.3.2 SPEED REGULATION

Figure 5.5 shows the forward speed during a cell-to-cell drive for each algorithm.

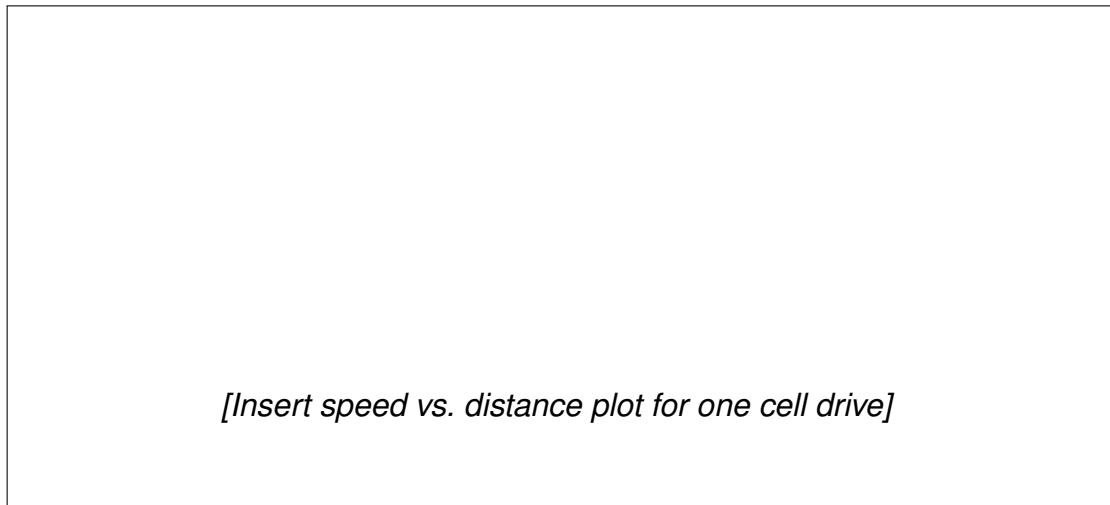


Figure 5.5: Forward speed regulation during cell-to-cell driving.

5.3.3 MAZE SOLVER PERFORMANCE

Table 5.2: Maze solver performance comparison.

Algorithm	Cells Explored	Total Time	Heading Error (avg)
PID	—	— s	—°
Fuzzy PI	—	— s	—°
Fuzzy Rule-Based	—	— s	—°

6. CONCLUSIONS AND PERSPECTIVES

6.1 CONCLUSIONS

This project demonstrated that fuzzy logic provides a viable and in several respects superior alternative to classical PID control for autonomous mobile robot navigation. By implementing and comparing three control strategies — PID, Fuzzy PI and Fuzzy Rule-Based — on the same hardware platform and the same physical tracks, the following conclusions can be drawn:

The PID controller offers the simplest implementation and the most predictable linear response. Its fixed gains produce a proportional steering command that is adequate for moderate disturbances, but leads to oscillatory behaviour near the lane centre (where the gains are too aggressive for the small error) and sluggish recovery at large disturbances (where the gains are insufficient).

The Fuzzy PI controller addresses this limitation by dynamically scaling the proportional and integral gains according to the magnitude of the disturbance. Near the centre, the gains are reduced by 20–30%, producing a smoother trajectory. Far from the centre, the gains are boosted by 20–30%, enabling faster recovery. The transition between regimes is smooth and continuous, governed by the overlapping triangular membership functions.

The Fuzzy Rule-Based controller eliminates the PI structure entirely and maps the disturbance and its rate of change directly to a turn rate through 15 human-readable rules. The addition of the trend input enables anticipatory control: the correction is moderated when the robot is already recovering, which eliminates the overshoot oscillations that affect both the PID and the Fuzzy PI. This controller achieved the smoothest trajectory in the lane keeping task.

For the maze solving task, all three algorithms successfully regulated the forward speed during cell-to-cell driving, with the DFS strategy ensuring complete exploration of the maze. The gyroscope-guided turning and side-sensor corridor centering proved essential for maintaining grid alignment over multiple successive turns.

The object-oriented architecture, designed around the SOLID principles, enabled the addition of new algorithms (Fuzzy PI, then Fuzzy Rule-Based) without modifying any existing controller code. The same algorithm classes were reused across both tasks by simply changing the configuration parameters — a practical validation of the dependency inversion and Liskov substitution principles.

6.2 SYNTHESIS OF CONTRIBUTIONS

The main contributions of this project are:

- Design and implementation of a dual-sensor lane keeping system with a three-tier safety architecture (emergency override, dead-band tolerance, algorithm regulation).

- Implementation and comparative evaluation of three control strategies (PID, Fuzzy PI, Fuzzy Rule-Based) on the same physical platform with quantitative metrics.
- Design of a 15-rule Mamdani fuzzy controller with two inputs (disturbance and trend) that achieves smoother trajectories than the PID through anticipatory correction.
- Geometric derivation of all membership function formulas from similar right triangles, providing an accessible mathematical foundation.
- Implementation of a DFS-based maze solver with gyro-guided turns, side-sensor centering, and trapezoidal distance integration for accurate cell-to-cell navigation.
- A SOLID-compliant object-oriented architecture that allows algorithms and tasks to be interchanged at runtime through a factory pattern.
- Real-time WiFi telemetry streaming for live monitoring and post-run quantitative analysis.

6.3 PERSPECTIVES FOR DEVELOPMENT

Several directions for future development arise from this work:

- **Automatic tuning:** the membership function boundaries and rule table were hand-tuned in this project. A genetic algorithm or particle swarm optimizer could be used to evolve optimal boundaries and rules automatically, using the Mean Absolute Disturbance as the fitness function.
- **Adaptive rules:** the Fuzzy Rule-Based controller uses a static rule table. An online learning mechanism (e.g. reinforcement learning on the rule consequents) could allow the controller to adapt its rules to changing track conditions during operation.
- **Maze solver path optimization:** the current DFS explores every cell but does not compute the shortest path. After exploration is complete, the maze map could be used to compute the optimal path using Dijkstra's or A* algorithm, and the robot could then drive the shortest route.
- **Computer vision:** replacing the colour and ultrasonic sensors with a camera would enable richer perception (recognizing intersections, signs, obstacles) and open the door to image-based fuzzy controllers.
- **Multi-robot coordination:** the WiFi streaming infrastructure could be extended to coordinate multiple robots exploring different sections of the maze simultaneously, partitioning the search space for faster coverage.

DECLARAȚIE DE AUTENTICITATE A LUCRĂRII DE FINALIZARE A STUDIILOR *

Subsemnatul **ION POPESCU**, legitimat cu **CI** seria **TZ** nr. **100772**, **CNP 5000102355779**, autorul lucrării **TITLUL LUCRĂRII DE FINALIZARE A STUDIILOR** elaborată în vederea susținerii examenului de finalizare a studiilor de **LICENȚĂ** organizat de către Facultatea **AUTOMATICĂ ȘI CALCULATOARE** din cadrul Universității Politehnica Timișoara, sesiunea **IUNIE** a anului universitar **2022**, coordonator **dr.ing. MIHAI IONESCU**, luând în considerare conținutul art. 34 din *Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație*, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale,
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului.
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență/diplomă/disertație.

Timișoara,

Data

Semnătura

*Declarația se completează „de mână” și se inserează în lucrarea de finalizare a studiilor, la sfârșitul acesteia, ca parte integrantă.